



Gestion dynamique des perturbations d'emploi du temps

Développement d'un prototype de gestion des perturbations d'emploi du temps sous contraintes avec CP-SAT (OR-Tools).



Stage de 4A Willing

Portapia Astrid (portapia@insa-toulouse.fr)

Encadrant (Willing): S. Figueroa-Iglesias (susely.figueroa-iglesias@willing.fr)

Encadrant (Willing): M. Veruete Villegas (mario.veruete@willing.fr)

Tuteur INSA: C. Dossal (dossal@insa-toulouse.fr)

Disclaimer

Avant de présenter ce rapport, je tiens tout particulièrement à remercier les personnes qui ont fait de ce stage une très bonne expérience.

Je veux tout d'abord exprimer ma gratitude à ma tutrice de stage au sein de Willing, Susely Figueroa-Iglesias. Elle a été disponible et d'une grande aide tout au long de mon stage. Je tiens aussi à remercier Mario Veruete Villegas, qui m'a aussi accompagnée et conseillée tout au long de cette expérience, ainsi que toute l'équipe de Coruscant qui a su m'accueillir avec beaucoup de bienveillance.

Je remercie également Messieurs Benjamin Provost et Nicolas Calmon pour leur confiance et pour m'avoir donné l'opportunité d'effectuer ce stage.

Finalement, je remercie tout le bureau Capitole de Willing qui m'a accueillie chaleureusement et avec qui j'ai passé de très agréables pauses et déjeuners.

Résumé à l'intention de lycéens

Pour des millions de collégiens, de lycéens et d'étudiants, un emploi du temps est un objet du quotidien. Mais sait-on vraiment à quel point il est difficile à construire ? Pour qu'il soit valide, il faut qu'il respecte de nombreuses contraintes : un professeur ne peut pas donner deux cours en même temps, une salle ne peut pas accueillir plus d'élèves qu'il n'y a de places, un groupe ne peut pas suivre deux cours simultanément... Et ce n'est qu'un début ! On souhaite qu'un emploi du temps soit valide, bien sûr, mais surtout qu'il soit **agréable** : éviter les fins de journées trop tardives, limiter les trous dans la journée, regrouper les cours dans le même bâtiment pour éviter de courir d'un bout à l'autre du campus...

Mais un emploi du temps, c'est aussi quelque chose de **dynamique**. Un professeur tombe malade, une salle devient indisponible, un cours doit être déplacé en urgence... Recréer entièrement l'emploi du temps à chaque imprévu est bien sûr impensable. Il faut donc savoir gérer ces *perturbations* intelligemment, en modifiant le moins de choses possible.

C'est exactement ce que j'ai exploré durant mon stage. En utilisant un outil mathématique appelé CP-SAT (un solveur de problèmes combinatoires), j'ai implémenté un générateur d'emplois du temps ainsi qu'un gestionnaire de perturbations. CP-SAT explore automatiquement des millions de combinaisons possibles pour trouver la meilleure solution, c'est bien plus efficace qu'une équipe humaine réunie autour d'une table !

Copyright

© [CC BY-NC-SA 4.0] Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International

Contact : portapia@insa-toulouse.fr

Table of Contents

1 Introduction	1
1.1 Présentation du problème	1
1.2 Etat de l'art	1
1.3 Mon stage	4
1.4 Plan	4
2 Prérequis scientifiques	5
2.1 CP-SAT	5
2.2 Algorithme glouton	6
2.3 Lexique	6
3 Génération d'emplois du temps	6
3.1 Modélisation mathématique du problème	6
3.2 Évolution de la modélisation	11
3.3 Implémentation	11
4 Gestion des perturbations	14
4.1 Modélisation et résolutions des perturbations	14
4.2 Implémentation du gestionnaire de perturbations	16
4.3 Résultats et limites	18
5 Intégration : pipeline complète	18
5.1 Difficultés rencontrées	18
5.2 Implémentation	19
5.3 Limitations actuelles	20
6 Conclusion et perspectives	20
6.1 Conclusion	20
6.2 Perspectives	20
Bibliographie	i



1 Introduction

1.1 Présentation du problème

Le problème d'emploi du temps universitaire (University Course Timetabling Problem, UCTP) consiste à affecter des cours à des créneaux temporels et à des salles tout en respectant un ensemble de contraintes strictes (contraintes dures) et en optimisant des préférences (contraintes souples). Ce problème est classiquement considéré comme NP-difficile en raison de la taille de l'espace de recherche et de la diversité des contraintes [6, 2]. Il est le sujet de grandes compétitions appelées International Timetabling Competition (ITC). Un autre thème associé à ce problème est la gestion des imprévus dans ces emplois du temps. En effet, il est tout bonnement impensable de recréer tout l'emploi du temps dès lors qu'une perturbation survient. Nous avons donc cherché les différentes méthodes qui sont mises en place pour répondre à l'UCTP, tout en gardant à l'esprit cette idée de gestion des perturbations.

1.2 Etat de l'art

Au fil des années, de nombreuses approches ont été proposées pour résoudre ce problème, allant des modélisations par graphes aux méthodes exactes, en passant par les métaheuristiques et, plus récemment, les approches hybrides.

Modélisation basée sur les graphes

Coloration de graphe (Graph Coloring) Une modélisation classique consiste à représenter chaque cours par un sommet de graphe. Une arête est ajoutée entre deux sommets si les cours correspondants sont en conflit, par exemple s'ils partagent des étudiants ou un enseignant. Le problème revient alors à colorer le graphe de sorte que deux sommets adjacents n'aient pas la même couleur, chaque couleur représentant un créneau horaire. Historiquement, la coloration de graphe est l'une des premières modélisations du timetabling car elle permet de représenter simplement les conflits entre cours. Bien que cette représentation soit souvent insuffisante pour modéliser des contraintes réelles complexes, elle demeure utilisée dans de nombreuses méthodes modernes comme étape de construction d'une solution initiale ou comme composant d'approches hybrides plus élaborées.

- *Avantages* : Modèle simple, intuitif et efficace pour des problèmes de base.
- *Inconvénients* : Difficulté à intégrer des contraintes complexes (préférences, contraintes spatiales ou temporelles fines).

Techniques de flot (Network Flow) Certaines approches modélisent des sous-problèmes d'affectation à l'aide de réseaux de flot, par exemple pour assigner des cours à des salles et des créneaux. Ces modèles sont généralement efficaces sur des versions simplifiées du problème, souvent résolubles en temps polynomial.

- *Avantages* : Efficacité pour les problèmes d'affectation structurés.
- *Inconvénients* : Incapacité à modéliser l'ensemble des contraintes globales du problème, nécessitant souvent des adaptations ou des itérations.



Méthodes exactes

Programmation Linéaire en Nombres Entiers (MIP / ILP) Dans cette approche, le problème est formulé à l'aide de variables binaires, de contraintes linéaires et d'une fonction objectif. Les solveurs MIP reposent sur des techniques comme la relaxation linéaire et le branch-and-bound pour explorer efficacement l'espace des solutions.

- *Avantages* : Permet d'obtenir des solutions optimales avec une preuve d'optimalité.
- *Inconvénients* : Modélisation complexe et explosion du nombre de variables lorsque les contraintes deviennent riches.

Les approches MIP ont montré leur efficacité sur des instances standardisées, notamment dans le cadre de la compétition ITC 2019, où des formulations avancées (par exemple basées sur des graphes de conflits) ont permis d'obtenir des résultats de pointe [3].

Programmation par contraintes et CP-SAT La programmation par contraintes (CP) repose sur une modélisation déclarative des contraintes et sur des mécanismes de propagation permettant de réduire l'espace de recherche. Le solveur CP-SAT d'OR-Tools constitue une évolution majeure de ces approches en combinant plusieurs paradigmes. Il repose sur une architecture hybride intégrant un moteur SAT (satisfaisabilité booléenne) utilisant des techniques modernes comme l'apprentissage de clauses (conflict-driven clause learning), des mécanismes de programmation par contraintes permettant la propagation de contraintes sur des variables entières, des techniques issues de la programmation linéaire, notamment des relaxations et des coupes pour guider la recherche.

- *Avantages* : Très grande flexibilité, modélisation naturelle des contraintes complexes, bonnes performances pratiques.
- *Inconvénients* : Difficulté plus grande à prouver l'optimalité complète et dépendance à la qualité de la modélisation.

Méthodes métaheuristiques

Recherche tabou La recherche tabou est une méthode de recherche locale qui explore itérativement le voisinage d'une solution tout en mémorisant les solutions récemment visitées afin d'éviter les cycles. Dans le contexte du timetabling, les mouvements de voisinage consistent généralement à déplacer un cours vers un autre créneau ou à échanger la position de deux cours. Cette capacité explique le succès historique de la recherche tabou, notamment dans les premières compétitions internationales de timetabling, notamment ITC 2007, où les approches de recherche locale et leurs variantes ont obtenu d'excellents résultats [5].

- *Avantages* : Bonne capacité d'exploration, robustesse.
- *Inconvénients* : Sensibilité aux paramètres et au choix du voisinage.

Algorithmes génétiques Les algorithmes génétiques reposent sur une population de solutions qui évoluent au fil des générations à travers des opérations de sélection, de croisement et de mutation. L'application des algorithmes génétiques au timetabling nécessite généralement de définir un codage adapté des emplois du temps sous forme de chromosomes. Les opérateurs de croisement et de mutation permettent alors de générer de nouvelles solutions en combinant ou modifiant des emplois du temps existants. Toutefois, ces



opérations produisent fréquemment des solutions violant certaines contraintes du problème, ce qui conduit souvent à intégrer des mécanismes de réparation ou des fonctions de pénalisation afin de maintenir la faisabilité des solutions [1].

- *Avantages* : Exploration globale de l'espace des solutions, flexibilité.
- *Inconvénients* : Convergence parfois lente et dépendance aux paramètres.

De manière générale, les métaheuristiques sont largement utilisées en pratique pour leur capacité à produire rapidement des solutions de bonne qualité sur des problèmes complexes.

Approches hybrides et matheuristics Les approches récentes de l'état de l'art reposent sur des méthodes hybrides, appelées *matheuristics*, combinant des solveurs exacts (MIP ou CP) avec des heuristiques ou métaheuristiques. Un exemple notable est la méthode *fix-and-optimize*, dans laquelle une partie des variables est fixée tandis qu'un solveur MIP optimise les variables restantes. Cette approche peut être vue comme une forme de Large Neighborhood Search (LNS) et permet d'explorer efficacement de larges portions de l'espace des solutions [4]. Dans le cadre de l'ITC 2019, plusieurs approches combinent MIP, heuristiques de réduction et méthodes d'amélioration locale afin d'obtenir des solutions de haute qualité [3]. D'autres travaux exploitent des stratégies de type LNS ou des approches hybrides combinant programmation par contraintes et recherche locale. Plus généralement, ces approches hybrides dominent désormais l'état de l'art, car elles permettent de tirer parti des avantages des différentes méthodes.

L'intérêt principal des matheuristics réside dans la combinaison structurée de méthodes exactes et heuristiques afin de guider efficacement l'exploration de l'espace des solutions. Ces approches permettent notamment d'améliorer la scalabilité des solveurs exacts sur des instances de grande taille, en orientant la recherche vers des régions prometteuses tout en conservant une structure globale cohérente du problème. Réciproquement, les heuristiques bénéficient de l'intégration de solveurs exacts, permettant d'obtenir des solutions de meilleure qualité et de structurer plus efficacement l'espace de recherche, notamment en optimisant localement certaines parties du problème.

Comparaison des approches Afin de synthétiser les principales caractéristiques des approches présentées dans cette section, le tableau A1 en annexe, que nous avons construit, compare la qualité des solutions, la flexibilité de modélisation, la capacité à traiter de grandes instances et la garantie d'optimalité. De plus, le tableau A2 en annexe, issu de notre propre synthèse, compare leurs forces et leurs limites dans le contexte du timetabling universitaire. Il est important de noter que CP-SAT peut prouver l'optimalité lorsqu'il termine la recherche complète, mais il est souvent utilisé comme solveur à temps limité dans les applications pratiques.

Analyse critique et choix méthodologique L'analyse de la littérature montre qu'aucune approche ne domine universellement le problème du timetabling universitaire, en raison de sa complexité et de la diversité des contraintes. Les approches MIP sont particulièrement efficaces sur des instances standardisées bien structurées, mais peuvent être difficiles à adapter à des contextes réels où les contraintes sont nombreuses et évolutives. À l'inverse, les approches basées sur la programmation par contraintes et les métaheuristiques sont largement utilisées en pratique, notamment dans des systèmes réels comme



UniTime, en raison de leur flexibilité. Enfin, les approches hybrides (matheuristics) constituent aujourd'hui l'état de l'art, en combinant la qualité des solutions produites par les méthodes exactes et la capacité d'exploration des heuristiques.

Au regard de cette analyse, nous choisissons d'explorer l'approche CP-SAT. En effet, la flexibilité d'ajout de contraintes dures et souples ainsi que la présence d'une preuve d'optimalité rend cette approche particulièrement adaptée à la génération des emplois du temps selon nous. De plus, cette flexibilité lui permet de fonctionner à la fois pour la génération et pour la gestion de perturbations. En effet, plutôt que de reconstruire entièrement l'emploi du temps après l'apparition d'une perturbation, on peut simplement fixer les sessions de cours non touchées et employer CP-SAT seulement sur les sessions impactées, ce qui est beaucoup plus efficace en pratique.

Solutions logicielles déjà existantes Nous avons aussi comparé des solutions logicielles déjà existantes, et surtout leur facilité d'utilisation et d'intégration, dans le tableau A3 en annexe. Nous nous rendons compte que ces logiciels ne proposent aucune gestion claire des imprévus et perturbations dans le contexte universitaire. UniTime est une solution logicielle open source que nous avons trouvée, elle semble être la plus complète de toutes car elle prend en compte des changements d'emploi du temps en plus de la génération. Cependant, la complexité de sa prise en main, de son déploiement et de sa configuration constitue une barrière importante à son adoption. De plus, les retours disponibles sur ces logiciels provenant essentiellement des éditeurs eux-mêmes, nous ne pouvons pas nous faire une véritable idée de la qualité de leurs services.

1.3 Mon stage

Le but de ce stage était d'obtenir un démonstrateur fonctionnel pour de la gestion de perturbations. Contrairement aux compétitions ITC, basées sur des instances standardisées, ce démonstrateur cible un usage réel avec des contraintes universitaires spécifiques, telles que des demi-groupes, qui sont absentes de ces formats.

Durant ce stage, nous avons pu créer via CP-SAT un générateur d'emplois du temps, ainsi qu'un gestionnaire de perturbations. Nous avons aussi réussi à lier les 2 via une pipeline de transformation de données.

1.4 Plan

Nous allons tout d'abord présenter plus en détails CP-SAT, la solution choisie, puis nous allons établir un lexique. Nous expliquerons ensuite précisément la construction et le fonctionnement de notre générateur d'emplois du temps, puis de notre gestionnaire de perturbations. Enfin, nous expliquerons la pipeline qui lie ces 2 modules. Nous terminerons par exposer des pistes d'avancées, notamment en IA.



2 Prérequis scientifiques

2.1 CP-SAT

Le solveur CP-SAT est composé et peut faire appel à de nombreuses méthodes :

La **propagation CP** permet d'éliminer dès le départ les combinaisons impossibles afin de ne pas avoir à tout tester (ex: si un professeur n'est pas là le lundi, on déplace tous les créneaux du lundi pour ce professeur). Cela permet donc de réduire l'espace de recherche. Le **SAT (SATisfaction booléenne)** teste des combinaisons, et quand une contradiction est trouvée, il cherche à "comprendre" pourquoi elle arrive et ajoute une règle générale pour éviter toutes les combinaisons similaires. SAT est donc très puissant car il *apprend* la structure du problème au fur et à mesure, donc devient plus intelligent, ce qui lui permet d'éliminer beaucoup de solutions à la fois, et d'éviter de refaire les mêmes erreurs.

Le **Branching heuristics** permet de choisir quoi tester, quelle variable ou quelle valeur tester en premier. Cela permet d'explorer moins de mauvais chemins. On peut par exemple choisir le cours le plus contraint ou la variable la plus critique. Cela permet un énorme gain de performance car on choisit les décisions les plus importantes en priorité, ce qui nous évite de visiter des branches de l'arbre de recherche qui sont "mortes", qui n'aboutissent pas.

Le **Restart** répond au problème qui apparaît souvent lors de la recherche d'une solution optimale : on se retrouve bloqué dans une mauvaise direction. Le restart permet de repartir de zéro dans notre recherche, tout en gardant en mémoire les apprentissages faits précédemment.

Le **Portfolio** permet au solveur CP-SAT d'utiliser plusieurs threads lors de sa recherche. Chaque thread travaille sur le même problème mais avec une stratégie et une heuristique différentes des autres. Les threads partagent leurs solutions ainsi que les contraintes qu'ils apprennent.

Le **LNS** est très important pour le timetabling. Cela consiste à "casser" une partie de la solution, réorganiser des cours au hasard par exemple, tout en ayant fixé une partie de la solution. Cela permet d'avoir une amélioration rapide et une exploration intelligente du domaine de recherche.

La **relaxation LP** ressemble essentiellement à du MIP. En effet, on regarde une version continue du problème, ce qui nous permet d'avoir une idée de ce qui est possible ou prometteur dans notre recherche. Cela permet donc de guider ladite recherche vers une direction intelligente.

Après sa recherche, CP-SAT renvoie un statut qui nous permet de connaître la qualité de la solution trouvée. OPTIMAL indique une solution qui remplit toutes les contraintes (dures et souples), FEASIBLE indique une solution qui remplit les contraintes dures mais pas entièrement les contraintes souples, UNKNOWN signifie que le solveur n'a pas eu le temps de trouver une solution durant le temps de recherche (ce dernier peut être manuellement adapté via la variable *timeout*) et INFEASIBLE indique que CP-SAT a trouvé qu'il était impossible de trouver une solution qui satisfait toutes les contraintes dures.



2.2 Algorithme glouton

Un algorithme glouton est un algorithme de placement. On lui donne une liste d'objets à placer, dans notre cas des sessions de cours, et il teste un à un les créneaux possibles pour chacun de ces objets. Dès lors qu'il trouve un créneau valide, qui respecte toutes les contraintes dures qu'on lui a données, il place l'objet à ce créneau, sans chercher à savoir si c'est le créneau optimal. Cela permet à cet algorithme de garantir une solution, contrairement à CP-SAT, mais elle peut n'être que partielle. En effet, le fait qu'il place séquentiellement les objets au premier créneau valide trouvé peut vouloir dire qu'il bloque des créneaux pour les objets suivants et donc peut empêcher complètement le placement de certains objets suivants. Un algorithme glouton donne donc toujours une solution, mais il est possible que certains objets à placer soit absents de cette solution. Un algorithme glouton est extrêmement rapide et respecte les contraintes dures, mais ne garantit pas une solution de qualité, ni le respect des contraintes souples.

2.3 Lexique

Nous appellerons par la suite:

- **instance** : l'ensemble des données d'entrée pour la création d'un emploi du temps (cours, professeurs, groupes, salles)
- **emploi du temps perturbé** : la solution trouvée par notre gestionnaire associée à un emploi du temps et une liste de perturbations donnée
- **greedy** : un algorithme glouton

3 Génération d'emplois du temps

Afin de mieux comprendre le problème des perturbations, nous avons décidé de commencer par comprendre comment un emploi du temps est construit. Pour ce faire, nous avons modélisé le problème ainsi que les contraintes que nous avons choisi d'implémenter.

3.1 Modélisation mathématique du problème

Ensembles et indices

- C : ensemble des cours
- T : ensemble des créneaux horaires
- R : ensemble des salles
- G : ensemble des groupes
- P : ensemble des professeurs
- D : ensemble des jours
- S : ensemble des créneaux dans un jour
- $C_g \subseteq C$: ensemble des cours du groupe g



- $C_p \subseteq C$: ensemble des cours du professeur p
- $L_d \subseteq T$: ensemble des créneaux de pause déjeuner du jour d

Modélisation des contraintes dures (ILP-like) Nous avons commencé par une modélisation qui semble assez intuitive : une qui est proche de l'Integer Linear Programming.

Variables de décision

$$X_{c,t,r} \in \{0,1\} \quad \forall c \in C, t \in T, r \in R$$

et on a :

$$X_{c,t,r} = \begin{cases} 1 & \text{si le cours } c \text{ est placé à la période } t \text{ dans la salle } r, \\ 0 & \text{sinon.} \end{cases}$$

Contrainte de non-chevauchement des groupes Un groupe ne peut avoir au plus qu'un cours par période

$$\sum_{c \in C_g} \sum_{r \in R} X_{c,t,r} \leq 1 \quad \forall g \in G, \forall t \in T$$

Contrainte de non-chevauchement des salles Une salle ne peut avoir au plus qu'un cours par période

$$\sum_{c \in C} X_{c,t,r} \leq 1 \quad \forall r \in R, \forall t \in T$$

Contrainte de non-chevauchement des enseignants Un professeur ne peut avoir au plus qu'un cours par période

$$\sum_{c \in C_p} \sum_{r \in R} X_{c,t,r} \leq 1 \quad \forall p \in P, \forall t \in T$$

Nombre de sessions Chaque cours doit être placé exactement le nombre de fois qu'il est prévu dans le syllabus

$$\sum_{t \in T} \sum_{r \in R} X_{c,t,r} = |c| \quad \forall c \in C$$

où $|c|$ est le nombre de sessions dans le cours c

Contrainte de capacité et de type de salle Une salle ne peut accueillir que des groupes qui ont un effectif inférieur ou égal à sa capacité. Une salle de labo ne peut pas accueillir un cours de CM/TD et une salle de CM/TD ne peut pas accueillir un cours de labo. Certains placements sont impossibles en raison des contraintes de capacité et de type de salle.

On impose :

$$X_{c,t,r} = 0 \quad \forall c \in C, \forall t \in T, \forall r \in R \quad \text{tels que :}$$

$$\text{capacity}(r) < \text{headcount}(\text{groupe}(c)) \quad \text{ou} \quad \text{type}(r) \notin \text{types autorisés de } c$$

Contrainte de pause déjeuner Cette contrainte garantit qu'il existe au moins un créneau libre dans la période de déjeuner.

$$\sum_{c \in C_g} \sum_{t \in L_d} \sum_{r \in R} X_{c,t,r} \leq |L_d| - 1 \quad \forall g \in G, \forall d \in D$$

Modélisation des contraintes souples (ILP-like) Soit w le poids associé à la contrainte décrite.


LongLunch Cas $|L_d| \leq 4$: minimisation des transitions cours/lunch

Nous introduisons une variable d'occupation agrégée :

$$occ_{g,t} = \sum_{c \in C_g} \sum_{r \in R} X_{c,t,r}$$

avec :

$$occ_{g,t} \in \{0,1\}$$

On introduit aussi des variables de transition :

$$\delta_{g,d,t} \in \{0,1\}, \quad \forall t \in L_d \setminus \{\min(L_d)\}$$

définies par :

$$\delta_{g,d,t} = 1 \iff occ_{g,t} \neq occ_{g,t^-}$$

où t^- désigne le créneau immédiatement précédent de t dans l'ordre de L_d .

La fonction objectif devient :

$$\min \sum_{g \in G} \sum_{d \in D} w \cdot \sum_{t \in L_d \setminus \{\min(L_d)\}} \delta_{g,d,t}$$

Cas $|L_d| > 4$: éviter une pause trop courte

On introduit une variable binaire :

$$\delta_{g,d} \in \{0,1\}$$

définie par :

$$\delta_{g,d} = 1 \iff \sum_{c \in C_g} \sum_{t \in L_d} \sum_{r \in R} X_{c,t,r} = |L_d| - 1$$

La fonction objectif devient :

$$\min \sum_{g \in G} \sum_{d \in D} w \cdot \delta_{g,d}$$

Cette modélisation distingue deux régimes :



- Pour des pauses déjeuner courtes, on impose une structure continue en minimisant les transitions entre périodes occupées et libres. Cela permet d'éviter des configurations fragmentées telles que libre/cours/libre au sein de la pause déjeuner.
- Pour des pauses plus longues, on relâche cette contrainte structurelle et on pénalise uniquement les pauses trop courtes (une seule période libre).

NoGap Soit $\delta_{g,d,t} \in \{0,1\}$, $\forall t \in T_\delta = T_d \setminus \{\min(T_d), \max(T_d)\} \setminus L_d$ défini par:

$$\delta_{g,d,t} = 1 \iff occ_{g,t-1} = 1 \text{ and } occ_{g,t} = 0 \text{ and } occ_{g,t+1} = 0$$

La fonction objectif est :

$$\min \sum_{g \in G} \sum_{d \in D} w \sum_{t \in T_\delta} \delta_{g,d,t}$$

NoLateDay La fonction objectif est:

$$\min \sum_{g \in G} \sum_{d \in D} w \sum_{t \in \{T^{--}, T^-\}} occ_{g,t}$$

où T^{--} et T^- sont respectivement l'avant dernier et le dernier créneau possible dans le jour d .

Closer Soit θ un seuil de distance donné et

$$d_{r,r'} = \begin{cases} 1 & \text{si } \text{dist}(\text{bat}(r_1), \text{bat}(r_2)) > \theta \\ 0 & \text{sinon.} \end{cases}$$

On introduit : $\delta_{g,d,t} \in \{0,1\}$, $\forall t \in T_\delta = T_d \setminus \{\max(T_d)\}$ défini par:

$$\delta_{g,d,t} = 1 \iff \sum_{c \in C_g} \sum_{r \in R} \sum_{c' \in C_g} \sum_{r' \in R} X_{c,t,r} \cdot X_{c',t+1,r'} \cdot d_{r,r'} \geq 1$$

La fonction objectif est :

$$\min \sum_{g \in G} \sum_{d \in D} w \sum_{t \in T_\delta} \delta_{g,d,t}$$

Chacune de ces contraintes souples a son pendant pour les professeurs. Dans la suite, nous indexons non plus par un créneau global t , mais par (d,s) . Ce choix est purement notationnel : $|T| = |D| \times |S|$, le nombre de variables reste identique.

Modélisation des contraintes dures (CP-like) Nous considérons un ensemble de cours C . Chaque cours $c \in C$ possède un nombre de sessions hebdomadaires S_c . On note R l'ensemble des salles, et $T = \{0, \dots, D \times S - 1\}$ l'ensemble des créneaux horaires, où D est le nombre de jours et S le nombre de créneaux par jour.



Variables Pour chaque session i du cours c , on introduit une variable entière :

$$slot_{c,i} \in \{0, \dots, |T| \cdot |R| - 1\}$$

Cette variable encode à la fois l'affectation temporelle et spatiale. On définit alors :

$$time_{c,i} = \left\lfloor \frac{slot_{c,i}}{|R|} \right\rfloor, \quad room_{c,i} = slot_{c,i} \bmod |R|$$

La variable $time_{c,i}$ correspond au créneau affecté, et $room_{c,i}$ à la salle utilisée pour la i^e session du cours c .

Contrainte de non-chevauchement des groupes Pour chaque groupe g , on note $sessions(g)$ l'ensemble des sessions associées aux cours suivis par ce groupe. On impose :

$$\forall g, \forall (c,i) \neq (c',j) \in sessions(g), \quad time_{c,i} \neq time_{c',j}$$

Contrainte de non-chevauchement des salles Afin d'éviter qu'une salle ne soit affectée à plusieurs cours au même moment, on impose que les variables $slot_{c,i}$ soient toutes distinctes :

$$\forall (c,i) \neq (c',j), \quad slot_{c,i} \neq slot_{c',j}$$

Contrainte de non-chevauchement des enseignants De manière analogue, pour chaque enseignant p , on note $sessions(p)$ l'ensemble de ses sessions :

$$\forall p, \forall (c,i) \neq (c',j) \in sessions(p), \quad time_{c,i} \neq time_{c',j}$$

Contrainte de non chevauchement des cours

$$\forall c \in C, \forall i \neq j, \quad time_{c,i} \neq time_{c,j}$$

Contrainte de capacité et de type de salle Pour chaque cours c et chaque session i , on impose que la salle choisie soit compatible :

$$\forall c \in C, \forall i \in \{1, \dots, S_c\}, \quad room_{c,i} \in R_c$$

où $R_c \subseteq R$ est l'ensemble des salles dont la capacité est suffisante et dont le type est compatible avec le cours c .

Contrainte de pause déjeuner On impose qu'au moins un créneau parmi les créneaux de midi reste libre pour chaque groupe et chaque jour.

Soit $L_d \subseteq T$ l'ensemble des créneaux correspondant au déjeuner au jour d . On impose alors :

$$\forall g, \forall d, \quad \sum_{(c,i) \in sessions(g)} \mathbf{1}_{time_{c,i} \in L_d} \leq |L_d| - 1$$

Cette contrainte garantit qu'au moins un créneau de midi reste libre pour chaque groupe.

Modélisation des contraintes souples (CP-like) Soit $sessions(g) = \{(c,i) \mid c \in C_g, i \in \{1, \dots, S_c\}\}$. On définit $occ_{g,t} = 1 \iff \exists (c,i) \in sessions(g) : time_{c,i} = t$.

Les modélisation des contraintes souples sont les même qu'en ILP-like, seule la définition de $occ_{g,t}$ diffère.



3.2 Évolution de la modélisation

Nous avons commencé par la formulation la plus naturelle : la formulation proche de l'ILP. Mais en testant sur une très grosse instance, nous nous sommes rendu compte de la faiblesse de cette formulation avec CP-SAT. En effet, la formulation initiale du problème reposait sur des variables binaires de la forme $X_{c,d,s,r}$, indiquant si un cours c est placé au jour d , au créneau s et dans la salle r . Cette modélisation, bien que correcte conceptuellement, entraîne une explosion combinatoire importante, puisque le nombre de variables est proportionnel à $|C| \times |D| \times |S| \times |R|$. Dans les grandes instances, cela conduit à des dizaines de millions de variables, rendant la création du modèle et sa résolution extrêmement coûteuses, voire impraticables. Ce type de formulation décrit explicitement toutes les combinaisons possibles et demande au solveur de sélectionner celles qui conviennent, ce qui est inefficace à grande échelle. Une approche plus adaptée consiste à utiliser une modélisation dite compacte, où chaque cours est associé directement à ses décisions de placement à l'aide de variables entières : une variable de temps indiquant le créneau affecté au cours, et une variable de salle indiquant la salle utilisée. Cela réduit drastiquement le nombre de variables (de l'ordre de $O(|C|)$ au lieu de $O(|C| \cdot |T| \cdot |R|)$), donc améliore sa scalabilité, et permet au solveur CP-SAT d'exploiter plus efficacement la structure du problème grâce à une meilleure propagation des contraintes. Nous avons maintenant la contrainte "Chaque cours placé le bon nombre de fois" qui est implicite puisque nous créons exactement le bon nombre de fois un cours directement.

Ainsi, le passage d'une modélisation exhaustive à une modélisation compacte est essentiel pour traiter des instances de grande taille de manière réaliste. Cependant, cette formulation peut encore générer des contraintes complexes, notamment pour garantir qu'une salle ne soit occupée que par un seul cours à un instant donné. Pour aller plus loin dans l'optimisation, il est possible de fusionner les variables de temps et de salle en une seule variable entière, appelée $slot_{c,i}$, définie comme une combinaison des deux dimensions (par exemple $slot = time \times |R| + room$). Cette variable encode directement un couple (créneau, salle) dans un seul entier.

Cette dernière formulation présente deux avantages majeurs. D'une part, elle réduit encore le nombre de variables et simplifie l'expression de certaines contraintes. D'autre part, elle permet d'utiliser des contraintes globales de CP-SAT telles que `AllDifferent`, qui imposent efficacement que deux sessions distinctes ne puissent pas occuper la même salle au même moment. Contrairement aux formulations précédentes basées sur des comparaisons pair à pair, cette approche exploite pleinement les mécanismes de propagation du solveur CP-SAT et améliore significativement les performances sur des instances de grande taille.

3.3 Implémentation

Premiers pas dans la génération Nous avons débuté par implémenter les contraintes dures que nous souhaitions avoir lors de la création de l'emploi du temps. Cela est fait directement dans la fonction `solve()`, avant l'appel au solveur CP-SAT. En effet, cela permet de réduire l'espace de recherche de ce dernier.

Nous avons fait le choix de commencer par un générateur hebdomadaire : on lui donne des groupes, des professeurs, des cours avec leur nombre de sessions par semaine, leur type de



salle (TD,CM,TP) ; et nous obtenons un emploi du temps. Nous l'affichons via une fonction qui crée un HTML à partir du fichier JSON de l'emploi du temps.

Nous avons très vite remarqué qu'en fonction des instances, nous obtenions des résultats plus ou moins satisfaisants. En effet, assez logiquement, plus une instance était simple (peu de sessions par semaine, beaucoup de salles, de professeurs, peu de cours...), plus on avait de chances d'obtenir une solution Optimale ; et au contraire, plus il y avait de tensions dans l'instance (peu de salles spécifiques, beaucoup de cours pour un seul professeur...), plus on obtenait des statuts UNKNOWN (même après une heure de recherche) ou même des statuts INFEASIBLE.

Le problème est double : une université réelle a généralement tendance à avoir beaucoup de tensions et il est complètement inacceptable de ne pas obtenir de solution lors de la génération d'un emploi du temps. C'est pourquoi nous avons décidé d'ajouter un filet de sécurité : un algorithme glouton (cf. Prérequis 2.2).

Une génération semestrielle Afin de gérer tous les types d'organisations universitaires, nous avons étendu notre pipeline CP-SAT+Greedy hebdomadaire en un générateur semestriel organisé en deux niveaux : répartition des sessions sur les semaines, puis résolution hebdomadaire.

Répartition des sessions L'idée de cette première phase est de trouver une répartition des sessions de cours sur tout le semestre selon les semaines. L'objectif étant d'avoir une répartition qui ne bloque pas la phase 2. Nous décidons donc de créer un autre solveur CP-SAT, `week_solve()`, avec des contraintes dures bien différentes de celles de la fonction `solve()` du générateur hebdomadaire. En effet, le but de ce solveur n'est pas de placer des tuples (session,groupe,professeur,salle) comme c'est le cas dans le solveur précédent, mais de placer les sessions dans les semaines du semestre de la manière la plus optimale possible. On cherche donc à ne pas concentrer tous les TPs sur une seule semaine, cela créerait des tensions pour les salles de TPs, mais aussi à ne pas mettre toutes les sessions d'un groupe durant une seule semaine (le groupe n'aurait pas le temps de suivre toutes ces sessions) par exemple. Finalement, nous avons 3 contraintes dures pour ce solveur : un nombre maximal de sessions par groupe par semaine, un nombre maximal de sessions par professeur par semaine, et le respect des semaines d'absence des professeurs, si tant est qu'elles ont été signalées précédemment.

Soit W l'ensemble des semaines sur lesquelles nous voulons créer l'emploi du temps, $week_{c,i} \in \{0, \dots, |W| - 1\}$ la semaine associée à la session i du cours c et $\forall p \in P$, A_p correspond aux semaines d'absence de professeur p . Nous avons donc les contraintes suivantes, dans l'ordre:

$$\forall g \in G, w \in W, \sum_{(c,i) \in \text{sessions}(g)} \mathbf{1}_{week_{c,i}=w} \leq M_g \quad \text{où} \quad M_g = \left\lceil 1,1 \cdot \frac{\max_{g' \in G} |\text{sessions}(g')|}{|W|} \right\rceil$$

$$\forall p \in P, w \in W, \sum_{(c,i) \in \text{sessions}(p)} \mathbf{1}_{week_{c,i}=w} \leq \left\lceil \frac{|\text{sessions}(p)|}{|W|} \right\rceil + 1$$

$$\forall c \in C_p, \forall i \in \{1, \dots, |c|\}, \forall w \in A_p, \quad \mathbf{1}_{week_{c,i}=w} = 0$$

où $|c|$ est le nombre de sessions du cours c



Nous avons choisi d'ajouter des contraintes souples, afin de rendre la répartition plus satisfaisante pédagogiquement. En effet, le professeur donne une répartition de ses sessions par cours, CM-CM-TD-CM-TD-TP par exemple, et si le solveur ne respecte pas cet ordre, il est pénalisé. De même, l'utilisateur peut donner une préférence temporelle : "early" s'il veut que les sessions de ce cours soient plutôt au début du semestre, "late" s'il préfère qu'elles soient vers la fin du semestre ou "middle" sinon. Enfin, nous avons ajouté une contrainte souple d'équilibre des sessions afin d'éviter d'avoir des semaines extrêmement chargées, et d'autres presque vides. Nous avons décidé d'implémenter cette dernière comme une contrainte souple et non une contrainte forte car cela permet de moins réduire l'espace de recherche, ce qui permet de diminuer les chances de statut INFEASIBLE du solveur. Voici les modélisations de ces contraintes souples, dans l'ordre :

Soit $\text{ord}_c = (i_1, i_2, \dots)$ l'ordre souhaité des sessions du cours c .

On définit $\delta_{c,k} = 1 \iff \text{week}_{c,i_k} > \text{week}_{c,i_{k+1}}$, et on minimise

$$\min w_{\text{ord}} \cdot \sum_c \sum_k \delta_{c,k} \quad \text{où } w_{\text{ord}} \text{ est le poids de cette contrainte.}$$

On minimise $w_{\text{pref}} \cdot \sum_{c \in C} \sum_{i=1}^{|c|} \text{pen}_{c,i}$ où $\text{pen}_{c,i} = \begin{cases} \text{week}_{c,i} & \text{si "early"} \\ |W| - 1 - \text{week}_{c,i} & \text{si "late"} \\ |\text{week}_{c,i} - \lfloor |W|/2 \rfloor| & \text{si "middle"} \end{cases}$

Soit $L_w = \sum_{c,i} \mathbf{1}_{\text{week}_{c,i}=w}$ et $\bar{L} = \left\lfloor \frac{n_{\text{total}}}{|W|} \right\rfloor$. On minimise $w_{\text{bal}} \cdot \sum_{w \in W} |L_w - \bar{L}|$

Résolution hebdomadaire Finalement, `week_solve()` crée des listes de sessions à placer par semaine, et il ne reste plus qu'à lancer la pipeline hebdomadaire CP-SAT+Greedy pour chaque semaine, en ne plaçant que les sessions qui lui sont attribuées. Si pour une semaine, le solveur CP-SAT `solve()` n'arrive pas à trouver une solution (UNKNOWN), nous avons décidé de relancer ce même solveur une seconde fois, mais sans les contraintes souples. Si là encore, nous n'obtenons pas de solutions (UNKNOWN ou INFEASIBLE), nous passons finalement à l'algorithme glouton. Cette pipeline hebdomadaire modifiée permet de limiter le recours au Greedy, et donc de favoriser les chances d'obtenir une solution proche de l'optimale. Elle est représentée dans la Figure 1 en annexe.

Résultats et limites Finalement, nous avons pu obtenir une pipeline semestrielle fonctionnelle : nous obtenons toujours une solution, et si certains cours n'ont pas pu être placés, ils sont gardés en mémoire dans une liste communiquée à l'utilisateur.

Nous avons testé notre générateur sur 3 instances synthétiques. L'instance `inge_semestre` est de loin celle qui a le planning hebdomadaire le plus chargé (2004 sessions sur tout le semestre, avec des professeurs qui ont jusqu'à 12 sessions par semaine). L'instance `polytech_s2` a la plus grande tension en termes d'affectation de salles car elle propose de nombreux types de salles différentes (TP Info, TP Physique, TP Chimie, TP Elec, TP MEca, TD, CM, etc.), mais peu de salles en soit : seulement 21. Enfin, l'instance `sorbonne_large_semestre` présente le plus de tension combinatoire : nous avons 311 cours pour 50 groupes.

Le tableau 1 présente les résultats obtenus dans des conditions identiques.

Nous en déduisons donc que notre générateur fonctionne généralement assez bien, même



Table 1: Comparaison des résultats de génération semestrielle

Instance	Statut réparation	OPTIMAL (/15)	FEASIBLE (/15)	GREEDY (/15)	Sessions non placées
inge_semestre	FEASIBLE	10	4	1	7 ($\approx 0,35\%$)
polytech_s2	FEASIBLE	15	0	0	0
sorbonne_large	FEASIBLE	15	0	0	0

sur d'assez grosses instances.

Nous avons cependant remarqué des limites. En effet, les semaines manquent généralement de diversité : nous pouvons retrouver des cours qui sont concentrés sur une ou deux semaines et n'ont presque plus aucune session dans le reste du semestre, comme observé dans la Figure 2 en annexe. C'est un assez gros problème car cela veut dire que l'emploi du temps généré ne respecte pas vraiment les attentes pédagogiques des universités. Une solution pour pallier ce problème serait d'ajouter une contrainte souple dans `week_solve()` (lors de la répartition des sessions sur les semaines) qui pousse à la diversité en pénalisant si le nombre de session d'un cours dans une semaine dépasse un certain seuil.

4 Gestion des perturbations

4.1 Modélisation et résolutions des perturbations

Nous avons choisi de modéliser les 9 perturbations qui nous paraissent les plus courantes et importantes. En effet, ce sont celles qui sont revenues le plus souvent lors de nos séances de questions avec des professionnels.

1 : Professeur Absent Nous avons nommé cette perturbation `teacher_absent()`. Elle a été implémentée dans l'idée d'une absence ponctuelle d'un professeur. En effet, elle vise à trouver les sessions touchées par les créneaux d'absence du professeur, et à les déplacer à d'autres créneaux. Cette perturbation interviendrait essentiellement lorsqu'un professeur est malade et ne peut assurer ses cours pour une semaine ou deux.

2 : Salle indisponible Nous avons nommé cette perturbation `room_unavailable()`. Elle permet de modéliser le cas où une salle ne peut plus être utilisée à des créneaux donnés, voire durant tout le semestre. Elle trouve les sessions impactées et les change de salle, sans les déplacer de jour ou de créneau.

3 : Libérer un créneau Nous avons nommé cette perturbation `free_slot()`. Elle permet de modéliser le cas où il faut libérer des créneaux donnés pour un ou plusieurs groupes, pour un événement par exemple. Cette perturbation trouve toutes les sessions des groupes impactés qui sont sur les créneaux donnés, et les déplace à d'autres créneaux disponibles.

Nous avons décidé d'implémenter un solveur et un algorithme glouton commun à ces trois perturbations. Chacune de ces perturbations peut appeler le même solveur CP-SAT, et si ce dernier donne un statut INFEASIBLE ou UNKNOWN, elle peut ensuite appeler le même



algorithme glouton.

4 : Remplacement de professeur Nous avons nommé cette perturbation `teacher_replacement()`. Elle est utile pour les absences prolongées de professeurs, qui entraînent un besoin de remplacement. L'utilisateur renseigne le professeur à remplacer ainsi que le cours, le type de session et le groupe touché. La perturbation utilise ces informations pour cibler les sessions de l'emploi du temps touchées et leur affecte un nouveau professeur en fonction de son quota d'heures. En effet, chaque professeur est d'un type donné, Maître de Conférence par exemple, et à ce type est lié un quota maximal d'heures de cours à donner. De plus, chaque professeur s'est donné une note d'adéquation aux différents cours, permettant donc à la perturbation de choisir le remplaçant le plus qualifié disponible pour la session. Cette perturbation possède un solveur CP-SAT et un algorithme glouton spécifiques.

5 : Déplacement de sessions Nous avons nommé cette perturbation `move_all()`. L'idée est simplement de déplacer une session donnée à un jour demandé, voire aussi à une heure de début demandée. Le déplacement change généralement la salle dans laquelle la session va se dérouler, mais l'utilisateur peut s'il le souhaite demander à garder sa salle originelle. Cette perturbation appelle la fonction `solve_move_all()`, qui cherche à résoudre tous les déplacements demandés via un solveur CP-SAT spécifique, mais si ce dernier échoue (INFEASIBLE ou UNKNOWN), elle appelle séquentiellement son greedy associé `move_onve()` sur tous les déplacements à faire. Si un déplacement échoue, la session à déplacer est simplement remis à sa place originelle.

6 : Permutation Cette permutation est nommée `permutation()`, et comme son nom l'indique, vise à permuter deux sessions données. Elle notifie tout d'abord l'utilisateur si les deux sessions n'ont pas la même durée. Elle vérifie ensuite qu'échanger les créneaux valides est possible. Si c'est le cas, elle déplace les sessions, si ce n'est pas possible, elle notifie quel échange de créneau est impossible. L'utilisateur peut choisir de déplacer au jour de permutation souhaité la ou les sessions qui n'ont pas pu être permutes. Si la permutation n'a pas abouti, et le déplacement le cas échéant, les sessions sont remises en place. C'est l'une des deux perturbations implémentées qui n'utilisent pas de solveur CP-SAT.

7 : Changement de salle Nous avons nommé cette permutation `all_room_change()`. Elle vise à changer la salle des sessions données. L'utilisateur peut donner une salle spécifique, ou simplement demander la meilleure salle selon les contraintes souples. Si une salle précise a été renseignée, la perturbation cherche à savoir si elle est disponible au jour et créneau de la session. Si c'est le cas, la session est déplacée dans cette salle, sinon la perturbation lance la fonction `solve_all_room_change()` qui contient un solveur CP-SAT spécifique. Ce dernier tente de placer toutes les sessions qu'on lui donne dans la meilleure salle possible selon les contraintes souples à chaque fois. S'il renvoie INFEASIBLE ou UNKNOWN, la perturbation appelle un algorithme glouton spécifique qui associe chacune des sessions demandées à la première salle disponible à son créneau.

8 : Ajout de sessions à un cours déjà existant Nous avons nommé cette perturbation `add_all_session()`. Elle vise à ajouter une session à un cours déjà existant. L'utilisateur doit renseigner le cours existant, le professeur qui donnera la session, le type de session et sa durée. Il peut aussi ajouter la semaine souhaitée, ainsi que le jour de la semaine et même l'heure de début souhaitée. Cette perturbation appelle la fonction `solve_all_add_session()`



qui contient un solveur CP-SAT spécifique. Si ce dernier donne un statut INFEASIBLE ou UNKNOWN, la perturbation fait appel à un greedy spécifique.

9 : Suppression de sessions Nous avons nommé cette perturbation `remove_all_sessions()`. Elle vise à retirer des sessions de l'emploi du temps. C'est la seconde perturbation qui n'utilise pas de solveur CP-SAT. En effet, le coeur de cette perturbation est simplement le filtrage selon les informations données par l'utilisateur pour trouver les sessions à supprimer. Une fois le filtrage complété, il suffit simplement de retirer les sessions, cela ne nécessite clairement pas CP-SAT.

Mécanismes importants lors de la résolution d'une perturbation Nous avons décidé d'ajouter quelques features qui nous paraissaient indispensables pour un bon gestionnaire de perturbations.

Groupes et sous groupes Nous avons mis en place une hiérarchie des groupes, leur permettant d'avoir un parent et des sous groupes. Cela permet de dédoubler des groupes pour des TP par exemple, ce qui est une pratique assez courante en université. Nous avons mis en place un filet de sécurité : un cours pour tout le groupe bloque le créneau pour tous les sous groupes, alors qu'un cours pour un sous groupe ne bloque le créneau que pour le groupe entier, et non pour les autres sous groupes.

Cours partagés Nous avons permis à l'emploi du temps d'avoir des cours partagés entre les groupes. Cela nous permet d'éviter de détecter des conflits fantômes et de gérer les perturbations intelligemment : si un cours partagé est déplacé à partir du moment où au moins un des groupes est indisponible.

Préférence de bâtiment Nous avons permis à l'utilisateur d'ajouter un bâtiment préféré pour chacun des cours. Cette idée nous est venue en réfléchissant à un emploi du temps multi-niveaux : il est courant de demander à ce que les étudiants en première année soient plutôt dans un bâtiment spécifique, alors que pour les étudiants plus âgés, non.

Mode semestriel Nous avons choisi de gérer les perturbations et les emplois du temps sous forme de jours absolus travaillés dans le semestre et non par couple (semaine,jour). Cela nous permet d'avoir un gestionnaire de perturbations qui fonctionne sur un jour, une semaine, un semestre voire même une année sans rien changer. Le seul bémol est que les jours à donner lors de la collecte des perturbations sont des jours absolus.

Dates limites Nous avons ajouté l'information du jour d'examen final pour chaque cours afin d'avoir une date limite de remplacement de cours. Le solveur est fortement pénalisé s'il replace une session après ou trop proche (seuil configurable via la variable `BUFFER_DAYS`) de l'examen final.

Gestion d'aujourd'hui Nous avons ajouté un jour absolu à saisir avant la collecte des perturbations, ce jour correspond à "aujourd'hui". Ce jour permet d'ignorer lors de la résolution toutes les sessions qui sont déjà passées, évitant de perdre du temps de calcul sur des cours ayant déjà eu lieu.

4.2 Implémentation du gestionnaire de perturbations

Données d'entrée et extraction Après quelques recherches, nous avons remarqué que la plupart des logiciels d'emplois du temps peuvent exporter lesdits emplois du temps sous



forme de CSV. Nous décidons donc de créer un fichier CSV comme emploi du temps initial. Nous nous basons sur un véritable emploi du temps pour écrire ce CSV, tout en anonymisant bien sûr. Nous implémentons aussi un fichier `specs.json` qui centralise toutes les données de l'université : bâtiments, salles, professeurs, cours et groupes.

A partir de ces deux fichiers, nous pouvons obtenir toutes les informations nécessaires pour la résolution des perturbations.

Collecte des perturbations L'utilisateur peut saisir autant de perturbations qu'il le souhaite, via la fonction `_collect_perturbations()`. Cette dernière lui permet de donner le type d'une perturbation et d'ensuite donner toutes les informations nécessaires pour avoir une perturbation complète. L'utilisateur peut à tout moment ordonner la résolution des perturbations saisies.

2 stratégies de résolution Nous avons décidé d'implémenter deux manières de résoudre un même problème de perturbation. La première est la résolution en cascade, c'est celle qui nous est venue en premier car elle est très utile pour simplement tester les résolutions des perturbations une à une. Nous avons appelé la seconde une résolution en une passe.

Résolution en cascade Comme son nom l'indique, la résolution en cascade est entièrement séquentielle. En effet, la résolution de chaque perturbation s'effectue à la suite de la précédente. La première perturbation a l'emploi du temps initial comme entrée et donne un emploi du temps solution, qui devient l'entrée de la perturbation suivante et ainsi de suite. Cette cascade permet de tester facilement les fonctions de résolution de toutes les perturbations. L'inconvénient est que c'est une résolution "bête" : elle traite chaque perturbation l'une à la suite de l'autre, sans réfléchir aux moyens d'optimiser la résolution.

Résolution en une passe C'est pour cela que nous avons décidé d'implémenter un autre type de résolution : celle en une passe. L'idée est de collecter toutes les perturbations, puis d'analyser chacune afin d'en déduire les sessions perturbées ainsi que les créneaux d'absence ou d'indisponibilité des professeurs, groupes et salles. Cela nous permet de regrouper les perturbations de même type ensemble, afin de les traiter en une fois. Au lieu de résoudre 3 fois une perturbation de Suppression de session, on traite une fois cette perturbation en lui donnant toutes les sessions à supprimer d'un coup par exemple. Après cette phase d'analyse, nous entrons dans la résolution à proprement parlé. Nous avons décidé de séparer la résolution, faite dans la fonction `full_solve()`, en 7 phases afin de la rendre la plus intelligente possible.

Nous avons regroupé les types de perturbations en fonction de leur degré d'importance, mais aussi en fonction de leur compatibilité et de leur impact sur l'emploi du temps. Nous avons décidé de résoudre en premier, la phase 0, la suppression de sessions car cela permet de "faire de la place" pour les autres résolutions. Par la suite, nous résolvons directement les perturbations les plus impactantes : professeur absent, salle indisponible et libérer créneau. Nous enchaînons ensuite avec la perturbation de remplacement de professeur car elle peut être très lourde. Nous continuons cette passe par les perturbations "de préférence", qui ont le moins d'impact : déplacer une session, permuter des sessions, changer de salle. Nous terminons la résolution par les ajouts de sessions. La Figure 3 en annexe représente le fonctionnement total de la résolution en une passe via `full_solve`.

Solution et affichage HTML Après la résolution, un résumé détaillé est donné dans la console et l'utilisateur peut choisir de générer le fichier HTML correspondant à l'emploi du



temps solution. S'il le fait, il a alors accès aux emplois du temps découpés en semaines pour tous les groupes, les salles ainsi que les professeurs. Tout échec d'ajout ou de suppression de sessions est listé dans un onglet rouge, et tout ajout ou suppression réussi est listé dans un onglet vert. Un fichier HTML récapitulant les heures de cours par professeur, notifiant tout changement dû aux perturbations, est aussi généré.

4.3 Résultats et limites

Nous avons testé notre gestionnaire de perturbations sur une instance CSV construite à partir d'un vrai emploi du temps : une semaine réelle répétée sur tout un semestre. Nous avons ajouté des examens finaux. Nous avons tenté différentes combinaisons de perturbations avec les 2 types de résolution. A chaque fois, la résolution en une passe a été très rapide : entre 10 et 20 secondes. La résolution en cascade est plus fluctuante en fonction du type de perturbations passées. Tout compte fait, notre module de gestionnaire de perturbations a très bien réagi à nos tests sur cette instance presque réelle. Cette dernière n'est cependant pas très grosse, il faudrait donc tester sur une instance réelle qui a beaucoup plus de groupes, de professeurs et de salles.

Le problème de ce module est qu'il ne sait gérer que des fichiers avec un format CSV très spécifique, qui peut ne pas être celui des exports des différents logiciels utilisés. Nous pourrions faire un parser qui transforme tout CSV d'export en un CSV compatible pour notre module.

5 Intégration : pipeline complète

Nous avons utilisé de nombreuses fonctionnalités de CP-SAT, mais il y en a une que nous n'avons jamais abordée : le hint, ou l'indice. L'idée est que l'on peut donner un hint à CP-SAT pour accélérer sa recherche de la solution optimale. En effet, on peut lui donner un point de départ spécifique lors de sa recherche au lieu de commencer au hasard, lui permettant de commencer sa recherche à un point plus avancé que sans le hint, si tant est que le hint soit véritablement de bonne qualité.

Nous avons réfléchi à une manière d'utiliser cette fonctionnalité au mieux et nous avons eu une idée : utiliser un MLP pour donner un point de départ au solveur CP-SAT. Ce concept fonctionnerait probablement si on entraînait ledit MLP sur des emplois du temps initiaux associés aux perturbations subies et à des solutions trouvées. Malheureusement, nous n'avons pas accès à ce genre de données. Nous décidons donc de créer une pipeline complète de génération/perturbations afin de créer ces données.

5.1 Difficultés rencontrées

Nos modules de génération et de gestion des perturbations possèdent d'assez grosses différences. En effet, le générateur ne sait pas gérer des demi groupes (il les voit simplement comme deux groupes à part entière), ni les cours partagés. Cependant, la plus grosse différence, et celle qui pose le plus de problème à l'élaboration de la pipeline, est le fait que



le générateur fonctionne en slots abstraits alors que le gestionnaire de perturbations fonctionne en heures/minutes. Il faudra donc transformer la sortie du générateur pour la rendre compatible avec le perturbateur. De plus, notre gestionnaire de perturbations fonctionne directement avec un CSV et non un emploi du temps sous forme de JSON. Nous devons donc le modifier légèrement pour qu'il puisse partir d'un emploi du temps en JSON.

5.2 Implémentation

Données d'entrée Nous avons décidé de séparer les informations de l'emploi du temps à générer en deux fichiers disjoints : `data_instance.json` et `specs.json`. Tandis que ce dernier permet d'avoir toutes les informations de l'université en elle-même (départements, bâtiments, salles, professeurs, groupes), `data_instance.json` permet d'avoir les informations précises pour l'emploi du temps : quel groupe suit quel cours, les types de salles pour les cours, les horaires pour manger, et bien d'autres.

Le générateur nécessitant une forme extrêmement précise pour ce fichier, nous avons décidé de créer un module `data_to_gen.py` qui permet de créer un fichier compatible avec le générateur appelé `instance_gen.json` à partir de `data_instance.json`.

Génération Pour faciliter la génération, nous avons créé une fonction `generate()` qui prend en entrée le nom de `data_instance.json`, le transforme en `instance_gen.json` via `data_to_gen.py`, et ensuite lance le main du module Générateur. Cela permet à l'utilisateur de saisir les informations importantes pour la génération telles que les poids des différentes contraintes souples pour la répartition des sessions sur chaque semaine. A la fin, nous obtenons un emploi du temps en slots abstraits : `edt_gen.json`.

Conformisation entre Générateur et Perturbations Malheureusement, cet emploi du temps n'est pas compatible avec ce qu'attend le module de Perturbation, notamment mais pas seulement car il est en slots abstraits. Nous avons donc implémenté le module `gen_to_perturb.py` qui permet de transformer `edt_gen.json` en un fichier compatible pour Perturbation appelé `edt.json`. Ce module requiert `specs.json` et `data_instance.json` en plus de `edt_gen.json` afin de pouvoir créer un `edt.json` beaucoup plus complet et informatif, ce qui est essentiel pour le module Perturbation.

Perturbation Le gestionnaire de perturbations tel que décrit dans la section 4.2 doit prendre un CSV comme emploi du temps. Nous avons donc décidé de ne pas l'utiliser tel quel dans cette pipeline, mais plutôt d'utiliser une version modifiée. Cela nous permet de conserver le gestionnaire fonctionnel, indépendant de la pipeline, mais aussi d'automatiser la gestion de perturbation dans la pipeline. En effet, nous avons implémenté le module `perturb.py` qui extrait de `edt.json` toutes les informations clés de l'emploi du temps, puis lance un gestionnaire de perturbations. A la fin, nous obtenons `edt_perturb.json`, l'emploi du temps solution des perturbations. Nous pouvons aussi récupérer le dict de chaque perturbation rencontrée grâce au fichier de log appelé `log_perturbations.jsonl`.

Pipeline complète et démo Nous avons donc un module `generate.py` qui permet de générer un emploi du temps à partir de `data_instance.json`, et un module `perturb.py` qui permet de collecter des perturbations et de les résoudre à partir d'un emploi du temps donné (`edt.json`) ainsi que les fichiers de données `data_instance.json` et `specs.json`. Finalement,



nous avons implémenté `pipeline.py` qui lance séquentiellement `generate.py` et `perturb.py`. Afin de tester toute la pipeline avec des réponses déjà préparées, nous avons aussi implémenté `demo.py`. Ce fichier nous permet de lancer plus facilement, et surtout plus rapidement, les tests. La pipeline complète est représentée par la Figure 4.

5.3 Limitations actuelles

Le générateur ne sachant pas gérer les demi groupes et les cours partagés, nous ne pouvons actuellement pas utiliser ces concepts dans nos emplois du temps, alors qu'ils sont très courants en université. De plus, le grand nombre d'appel à l'utilisateur tout au long de la pipeline la rend longue à faire tourner. Le module `demo.py` atténue ce problème en permettant de préparer les réponses à l'avance, mais il faut être très rigoureux dans sa création. Enfin, un seul couple (`data_instance.json`, `specs.json`) limite la diversité des données d'entraînement : il faudrait des données provenant de plusieurs universités pour entraîner un MLP robuste.

6 Conclusion et perspectives

6.1 Conclusion

Au cours de ce stage, nous avons étudié différentes méthodes de résolution de l'UCTP et compris l'utilisation du solveur CP-SAT. De plus, nous avons implémenté deux modules indépendants fonctionnels qui répondent chacun à une partie du problème : génération et gestion des perturbations. Nous avons aussi créé une pipeline rudimentaire liant ces deux modules qui produira les triplets (emploi du temps initial, perturbations, emploi du temps solution) nécessaires à l'entraînement d'un MLP dans le but d'instaurer un warm start pour la résolution de perturbations.

6.2 Perspectives

Dans le futur, nous devons entraîner ledit MLP et mettre en place le warm start. Un modèle de type JEPA (Joint Embedding Predictive Architecture) pourrait être plus adapté qu'un MLP classique, car il apprend des représentations latentes de la structure du problème plutôt que prédire directement les affectations. Nous envisageons aussi d'intégrer de l'Inverse Reinforcement Learning (IRL) : en résolvant un même problème avec différents poids pour les contraintes souples et en laissant l'utilisateur choisir sa solution préférée, il serait possible d'apprendre directement les poids idéaux pour ce dernier. Enfin, une interface utilisateur pour nos modules est indispensable, permettant la saisie des informations pour la génération et la perturbation. Cette saisie pourrait se faire via des cases à cocher, ou en phrases descriptives, un petit LLM se chargeant de les transformer en entrées structurées pour les modules.



Bibliographie

- [1] Edmund K Burke, David Elliman, and Rupert Weare. "A genetic algorithm based university timetabling system". In: *Proceedings of the 2nd east-west international conference on computer technologies in education*. Vol. 1. 1994, pp. 35–40.
- [2] Mohammad-Reza Feizi-Derakhshi, Hamed Babaei, and Javad Heidarzadeh. "A Survey of Approaches for University Course Timetabling Problem". In: *Proceedings of 8th International Symposium on Intelligent and Manufacturing Systems*. Adrasan, Antalya, Turkey: Sakarya University Department of Industrial Engineering, 2012, pp. 307–321.
- [3] Dennis Søren Holm et al. *A MIP Based Approach for International Timetabling Competition 2019*. 2020.
- [4] Michael Lindahl, Matias Sørensen, and Thomas R. Stidsen. "A Fix-and-Optimize Matheuristic for University Timetabling". In: *Journal of Heuristics* 24.4 (2018), pp. 645–665.
- [5] Zhipeng Lü and Jin-Kao Hao. "Adaptive tabu search for course timetabling". In: *European journal of operational research* 200.1 (2010), pp. 235–244.
- [6] Andrea Schaerf. "A Survey of Automated Timetabling". In: *Artificial Intelligence Review* 13.2 (1999), pp. 87–127.



Appendices

Table A1: Comparaison des principales approches pour le problème de timetabling universitaire

Approche	Qualité de solution	Flexibilité	Mise à l'échelle	Optimalité garantie
Coloration de graphe	Moyenne	Faible	Excellente	Non
MIP / ILP	Excellente	Moyenne	Moyenne	Oui
CP-SAT	Très bonne	Excellente	Bonne	Partielle
Recherche tabou	Bonne	Excellente	Très bonne	Non
Algorithmes génétiques	Bonne	Excellente	Très bonne	Non
Matheuristics	Très bonne à excellente	Excellente	Très bonne	Généralement non

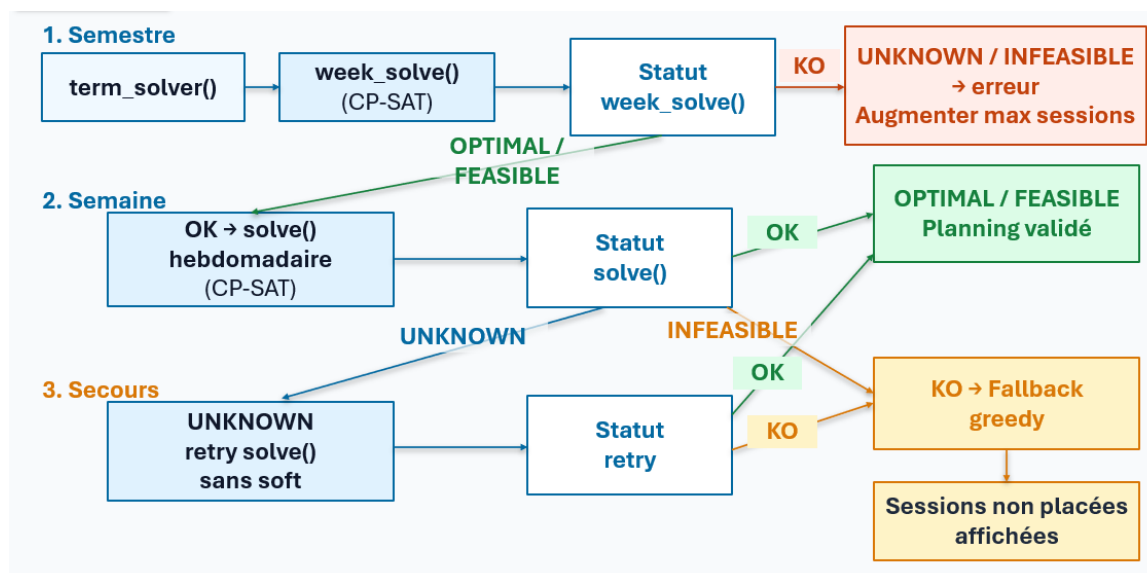


Table A2: Forces et limites des principales approches de résolution du problème de timetabling universitaire

Approches	Forces	Limites
Coloration de graphe	Modélisation simple et intuitive des conflits entre cours. Faible coût de calcul et utilisation fréquente pour construire des solutions initiales.	Représentation limitée des contraintes réelles (salles, préférences, disponibilités, contraintes temporelles complexes).
Programmation linéaire en nombres entiers (MIP / ILP)	Obtention de solutions optimales avec preuve d'optimalité . Cadre mathématique rigoureux et largement étudié.	Explosion du nombre de variables et du temps de calcul lorsque la taille du problème ou le nombre de contraintes augmente.
Programmation par contraintes (CP-SAT)	Modélisation naturelle des contraintes complexes . Grande flexibilité et excellentes performances pratiques sur de nombreux problèmes combinatoires.	Performances fortement dépendantes de la qualité de la modélisation et du paramétrage de la recherche .
Recherche tabou	Bonne capacité à échapper aux optima locaux grâce à la mémoire tabou. Méthode robuste sur des instances de grande taille.	Nécessite un choix pertinent du voisinage et un réglage soigné des paramètres .
Algorithmes génétiques	Exploration globale de l'espace des solutions. Adaptables à des contraintes variées et facilement hybridables avec d'autres méthodes.	Convergence parfois lente et génération fréquente de solutions infaisables nécessitant des mécanismes de réparation .
Approches hybrides (matheuristics)	Combinaison des avantages des méthodes exactes et heuristiques. Permettent d'obtenir des solutions de haute qualité sur des instances complexes.	Conception, implémentation et paramétrage plus complexes que les approches classiques.


Table A3: Comparaison des modèles d'intégration

Logiciel	Modèle d'intégration
ADE Campus	Basé sur la maquette pédagogique. Connecteur générique : SQL ou CSV, possibilité d'ajouter des données à la main
HyperPlanning	Importation du planning depuis un tableur, basé sur la maquette pédagogique
Celcat Timetabler	Accent sur la facilité d'intégration avec les systèmes d'information de l'enseignement supérieur. Construit sur une base de données orientée graphe
Google Calendar	Intégration complète avec l'écosystème Google (exemple : détection de nouveaux RDV dans les mails)
Microsoft Bookings	Intégration complète avec l'écosystème Microsoft


Figure 1: Pipeline de génération semestrielle


Groupe 1A_A — Semaine 7

	Lundi	Mardi	Mercredi	Jeudi	Vendredi
t1	C3 Dubois TD_101	C4 Thomas TD_101	C5 Robert Amphi_A		
t2	C4 Thomas TD_101	C5 Robert Amphi_A	C5 Robert TD_103	C8 Durand TD_102	
t3	C5 Robert Amphi_A			C5 Robert Amphi_A	
t4 (Pause)			C6 Richard Amphi_A		
t5 (Pause)					
t6	C8 Durand TD_101	C5 Robert Amphi_A		C4 Thomas TD_101	
t7					
t8					

Groupe 1A_A — Semaine 8

	Lundi	Mardi	Mercredi	Jeudi	Vendredi
t1	C6 Richard Amphi_A	C5 Robert TD_101	C5 Robert TD_101	C5 Robert TD_101	C5 Robert Info_101
t2	C6 Richard Amphi_A	C6 Richard Info_101	C7 Petit TD_101	C8 Durand TD_101	C5 Robert TD_101
t3	C6 Richard Amphi_A	C5 Robert TD_101		C5 Robert TD_101	
t4 (Pause)					
t5 (Pause)					
t6	C4 Thomas TD_101	C2 Bernard TD_101		C7 Petit TD_101	C6 Richard Amphi_A
t7					
t8					

Figure 2: Semaines 7 et 8 d'un emploi du temps généré

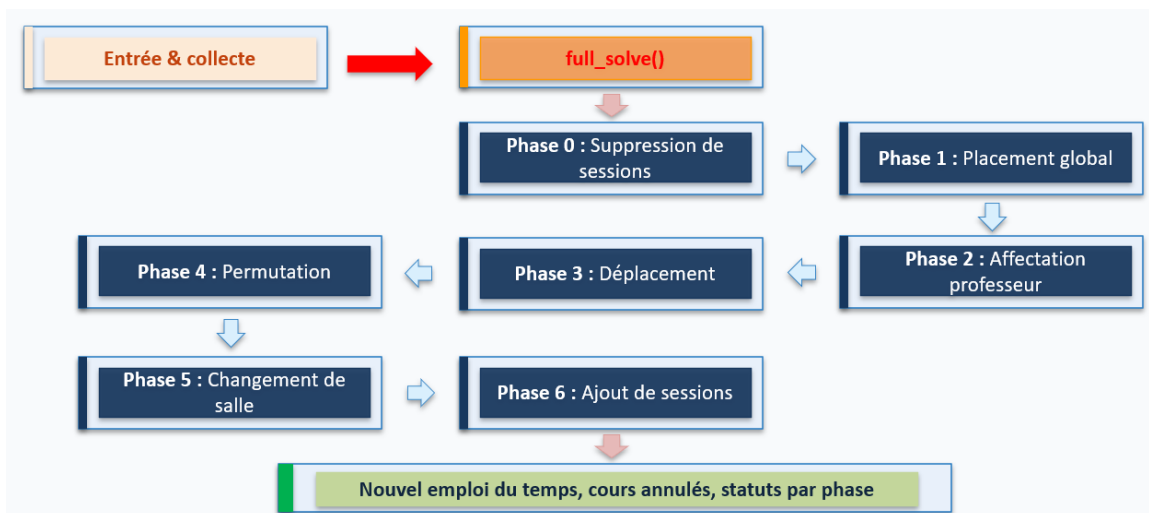


Figure 3: Phases de la résolution en une passe

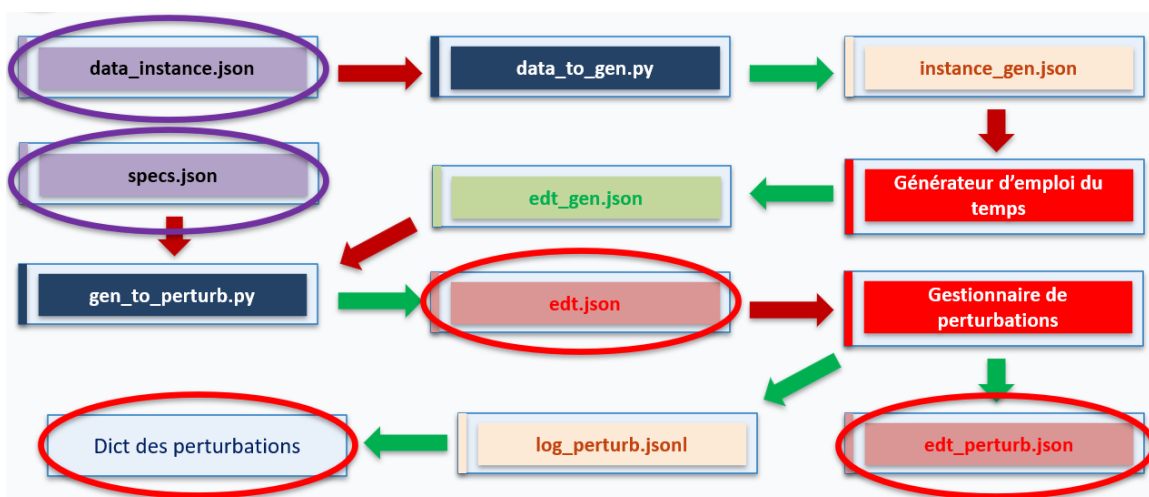
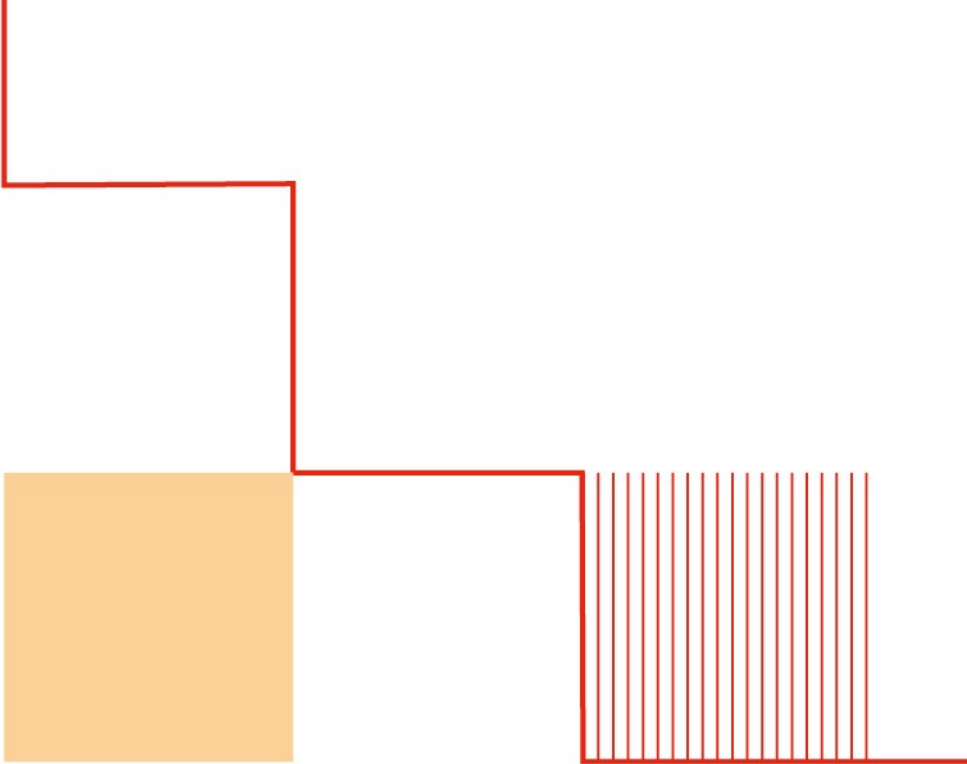
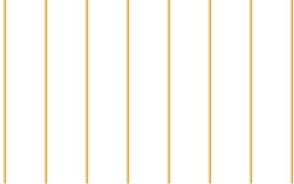


Figure 4: Pipeline de génération de données

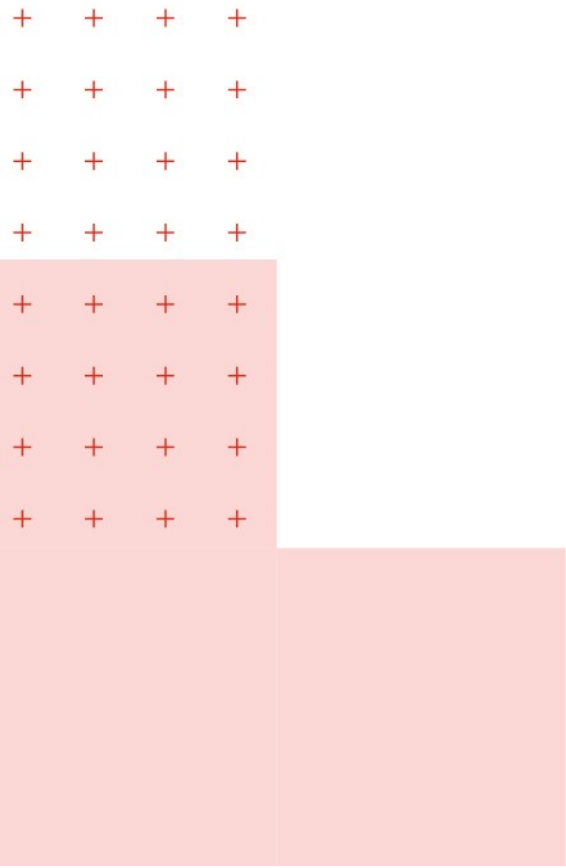


INSA TOULOUSE

135 avenue de Rangueil
31400 Toulouse

Tél : + 33 (0)5 61 55 95 13

www.insa-toulouse.fr



INSA | INSTITUT NATIONAL
DES SCIENCES
APPLIQUÉES
TOULOUSE